NEUF CHALLENGES · NEUF DÉCLICS

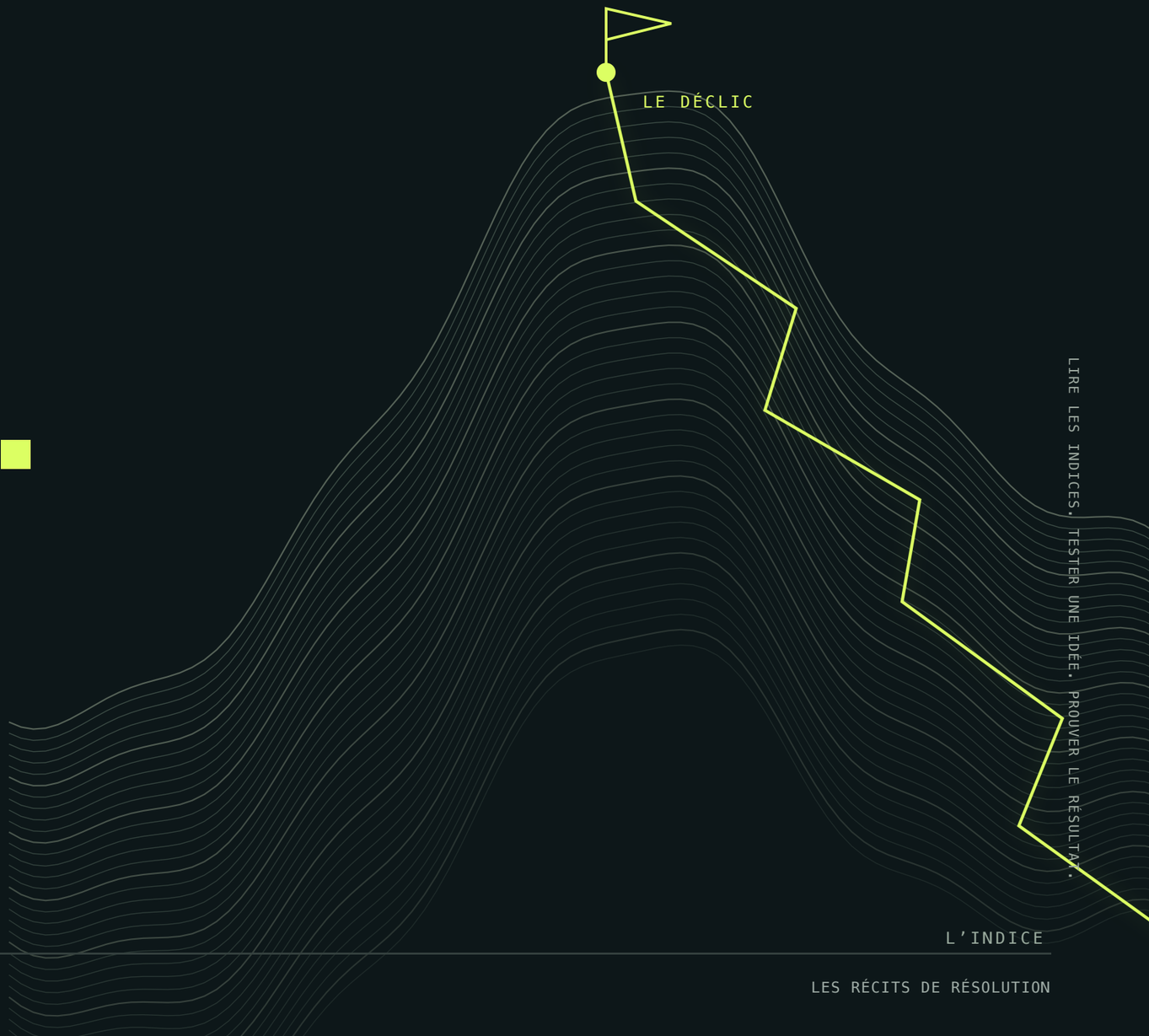
DERRIÈRE LES FLAGS.

Les agents ont trouvé les failles.
À nous de comprendre le chemin.

01

COMMENÇONS PAR UNE IDÉE SIMPLE

lost_in_embedding →



LE DÉFI · RETROUVER UN TEXTE CACHÉ

Le flag avait disparu.
Il restait des nombres.

FICHIER FOURNI

Dockerfile.json

```
model: "gpt2"
inputs_embeds:
  [ 0.1145, -0.3451, 0.2239, … ]
  [ … ]
  …
```

Extrait arrondi des premières valeurs.

24 groupes de nombres

768 nombres par groupe

Chaque groupe représente un morceau du texte, avec une information sur sa position.

L'IDÉE À RETENIR

Notre mission : **reconstituer le texte d'origine.**

COMPRENDRE · QU'EST-CE QU'UN EMBEDDING ?

Un morceau de texte.
Une ligne dans une table.

MORCEAU DE TEXTE

BA

Un « token »

→

TABLE PUBLIQUE DE GPT-2

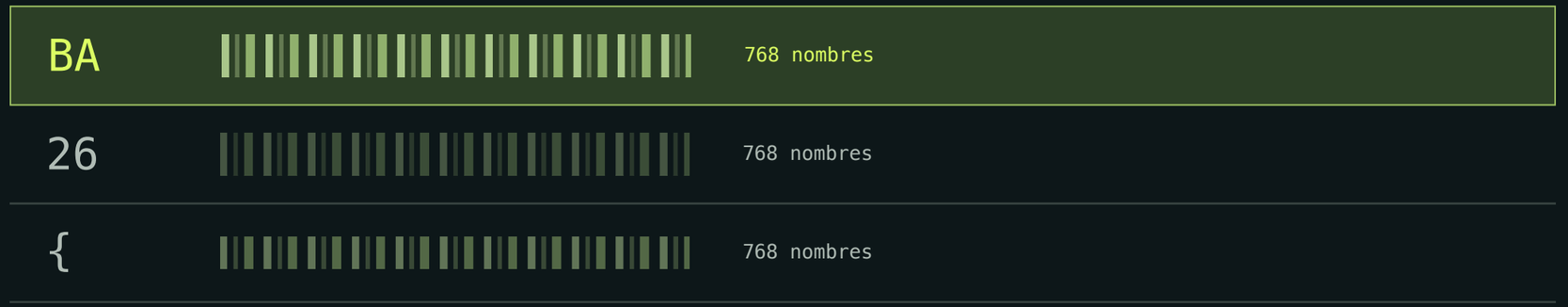


Schéma : les barres représentent des listes de nombres.

Dans ce fichier, on ajoute aussi : LA POSITION DU MORCEAU

Pour distinguer le premier, le deuxième, le troisième…

L'IDÉE À RETENIR

Le morceau de texte et sa position viennent de **deux tables connues.**

LE DÉCLIC · ON CONNAÎT LES DEUX TABLES

Retirer la position.
Retrouver le morceau.

01

Lire un groupe

nombres du morceau

+ nombres de sa position

Les nombres reçus.

02

Retirer la position

nombres du morceau

~~nombres de sa position~~

Elle est dans la table publique.

03

Comparer à la table

BA ✓

Le morceau le plus proche est retrouvé.

L'IDÉE À RETENIR

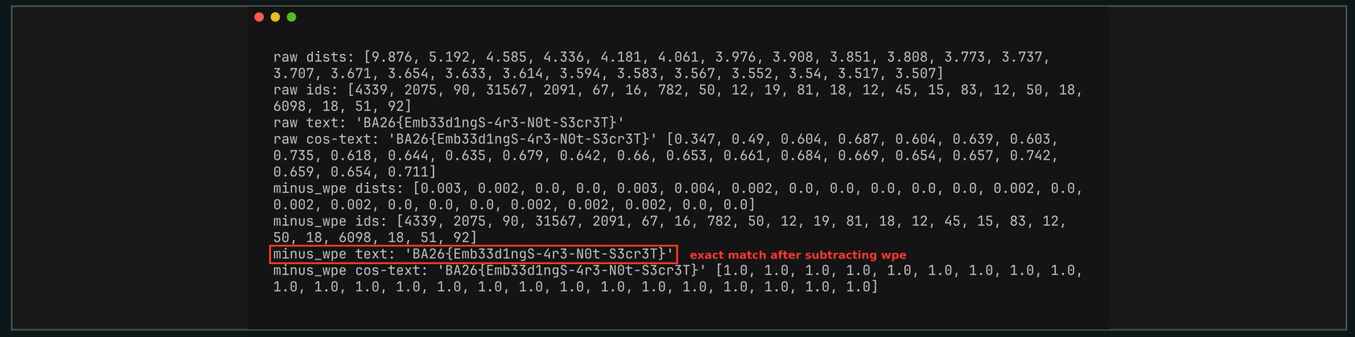
Recommencer pour les **24 groupes**, puis assembler les morceaux.

LA PREUVE · LES 24 MORCEAUX S'ASSEMBLENT

Les nombres ont retrouvé leurs lettres.

BA	26	{	Emb	33	d	1	ng	S	…	}

BA26{Emb33d1ngS-4r3-N0t-S3cr3T}



```
raw dists: [9.876, 5.192, 4.585, 4.336, 4.181, 4.061, 3.976, 3.908, 3.851, 3.808, 3.773, 3.737,
3.707, 3.671, 3.654, 3.633, 3.614, 3.594, 3.583, 3.567, 3.552, 3.54, 3.517, 3.507]
raw ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12, 50, 18,
6098, 18, 51, 92]
raw text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'
raw cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [0.347, 0.49, 0.604, 0.687, 0.604, 0.639, 0.603,
0.735, 0.618, 0.644, 0.635, 0.679, 0.642, 0.66, 0.653, 0.661, 0.684, 0.669, 0.654, 0.657, 0.742,
0.659, 0.654, 0.711]
minus_wpe dists: [0.003, 0.002, 0.0, 0.0, 0.003, 0.004, 0.002, 0.0, 0.0, 0.0, 0.0, 0.0, 0.002, 0.0,
0.002, 0.002, 0.0, 0.0, 0.0, 0.002, 0.002, 0.002, 0.0, 0.0]
minus_wpe ids: [4339, 2075, 90, 31567, 2091, 67, 16, 782, 50, 12, 19, 81, 18, 12, 45, 15, 83, 12,
50, 18, 6098, 18, 51, 92]
minus_wpe text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}'   exact match after subtracting wpe
minus_wpe cos-text: 'BA26{Emb33d1ngS-4r3-N0t-S3cr3T}' [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

24 / 24

Morceaux retrouvés.
Correspondances presque exactes
après retrait de la position.

Solve : Claude

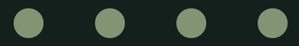
L'IDÉE À RETENIR

Dans ce cas, transformer le texte en nombres **ne protégeait pas son contenu.**

LE DÉFI · ACCÉDER AU COMPTE ADMINISTRATEUR

Un compte admin. Deux portes d’entrée.

CONNEXION CLASSIQUE



Email + mot de passe

Le mot de passe de l’admin reste inconnu.

CONNEXION GOOGLE

ATTESTATION D’IDENTITÉ

Numéro du compte

« Voici qui se connecte. »

Une attestation transmise au site

Le site lit le numéro du compte.

LA FAILLE

Sur le chemin « legacy », **le site ne vérifiait pas la signature.**

LES IMPASSES · CHANGER LE RÔLE NE SUFFIT PAS

Écrire « admin » ne rendait pas admin.

✕ **Ajouter « rôle : admin »**
Le site ignore cette demande.
UTILISATEUR

✕ **Copier l'email de l'admin**
Avec un autre identifiant, cela crée un autre compte.
UTILISATEUR

LE CHAMP QUI COMPTE

`sub`

Le numéro qui désigne **un compte Google précis.** → Retrouver le compte existant, avec son rôle enregistré.

L'IDÉE À RETENIR

Il fallait retrouver **l'identifiant du vrai compte admin.**

LE DÉCLIC · L’INDICE ÉTAIT AU PIED DE LA PAGE

Le numéro était trouvable.
Le site le traitait comme un secret.

01

Le pied de page

Tech Corp Chandler

Une adresse Gmail de contact

Un indice sur l’identité de l’admin.

02

La recherche du compte

Google People API

Avec une session Google de test

L’adresse permet ici de retrouver un identifiant.

03

Le bon numéro

`1034…4574`

GAIA ID / sub · abrégé

À placer dans l’attestation fabriquée.

L’IDÉE À RETENIR

Le déclic : **chercher l’identité** au lieu de deviner un mot de passe.

LE RÉSULTAT · SOLVE CONSIGNÉ À 21:25

Le bon identifiant.
Une session administrateur.

Attestation fabriquée → Compte existant retrouvé → ADMIN

EXTRAIT DU WRITE-UP CONSERVÉ · TRANSCRIPTION

socialLogin	memberId: 1 · role: admin
/api/flag	flag retourné

Le endpoint « /api/admin/flag » était un leurre : même connecté, il renvoyait 404.

BA26{sub_is_not_a_secret_but_treating_it_like_a_password_is}

L'IDÉE À RETENIR

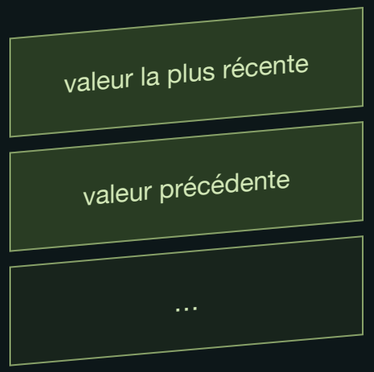
Vérifier **la preuve d'identité**, pas seulement lire le numéro.

LE DÉFI · COMPRENDRE UNE MACHINE INCONNUE

Le programme est là. Son dictionnaire manque.

DES CODES, MAIS QUELLES ACTIONS ?

```
03  →  ?
04  →  ?
19  →  ?
```



Une « machine à pile »

Elle range des valeurs en pile, puis les copie, les déplace ou fait des calculs.

Le coup de pouce : 4 programmes d'exemple + leurs résultats attendus

L'IDÉE À RETENIR

Les exemples vont permettre de **retrouver le sens des codes.**

LE DÉCLIC · UN DICTIONNAIRE DOIT TOUT EXPLIQUER

Essayer une traduction.
La confronter aux exemples.

01

Attribuer un sens

Quel code veut dire « copier », « afficher », « additionner »… ?

→

02

Simuler la machine

Faire passer les exemples dans cette interprétation.

→

03

Comparer les sorties

Éliminer les dictionnaires qui donnent un autre résultat.

4 EXEMPLES CONNUS → **1** INTERPRÉTATION QUI LES SATISFAIT TOUS

L'IDÉE À RETENIR

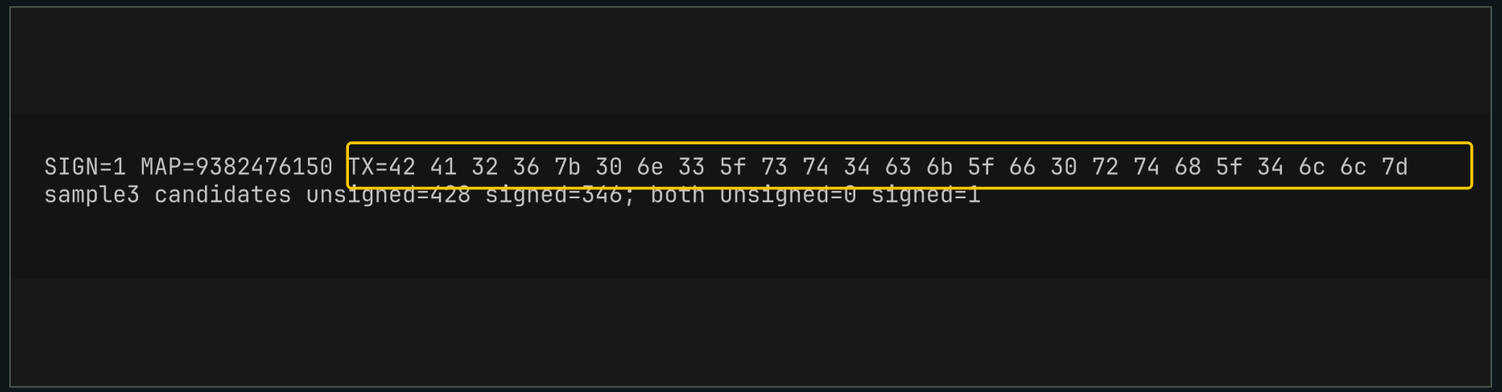
Une bonne interprétation doit réussir **tous les tests fournis.**

LA PREUVE · LE DICTIONNAIRE RETROUVÉ FONCTIONNE

La machine retrouve sa langue. Le message devient lisible.

03	copier le sommet
04	émettre un octet
19	additionner

Trois exemples du dictionnaire retrouvé.
Codes écrits en hexadécimal.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Solve : **Codex** · Sortie enregistrée du solveur.

`BA26{0n3_st4ck_f0rth_4ll}`

L'IDÉE À RETENIR

Les résultats connus ont servi de **mode d'emploi pour lire l'inconnu.**

LE DÉFI · OUVRIR UN ENREGISTREMENT PROTÉGÉ

Le programme d’une balise.
Un secret derrière un code.

FICHIER FOURNI

`beacon_flash.bin`



Une copie de la mémoire
d’un appareil ESP32-C6.

→

CE QUE LE PROGRAMME VÉRIFIE

Code d’activation ?

_ _ _ _ _ _ _ _

Le bon code autorise
le décodage de l’enregistrement.

LE PIÈGE ~~`BA26{n0t_th15_0n3_k33p_d1gg1ng}`~~ Présent en clair, mais ce n’est pas le résultat recherché.

L’IDÉE À RETENIR

Trouver un texte qui ressemble à un flag **ne suffit pas à prouver le solve.**

LE DÉCLIC · LA RECETTE EST DANS LE PROGRAMME

Le programme contenait la recette du code.

DEUX TEXTES DANS LE FICHIER

`LilyGO T7-C6`

`BA26-BEACON-01`

→

01

Transformer et combiner

Reproduire les opérations que fait le programme.

→

CODE ATTENDU

B80FEF82

Retrouvé par l'agent.

ENSUITE **Reproduire le décodage** → Lire les 31 octets de l'enregistrement.

L'IDÉE À RETENIR

L'agent a **reproduit le calcul** : pas besoin de tester tous les codes.

LA PREUVE · L'ENREGISTREMENT DÉCODÉ

Suivre la bonne branche. Lire le vrai enregistrement.

CODE RETROUVÉ

B80FEF82 ✓

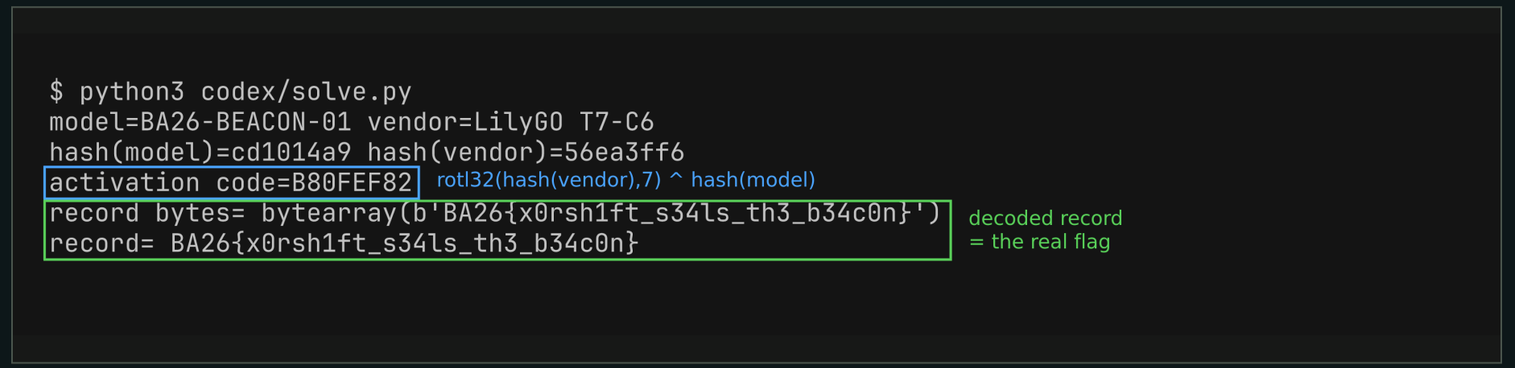
Calcul du code

↓

Décodage des données

↓

Enregistrement lisible



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Solve : **Codex** · Résultat du solveur hors ligne.

`BA26{x0rsh1ft_s34ls_th3_b34c0n}`

L'IDÉE À RETENIR

La preuve vient du **chemin de décodage**, pas d'un texte trouvé au hasard.

LE DÉFI · UNE APPLICATION DE FICHIERS

On demande un fichier. Le serveur suit le chemin.

LE DOSSIER PRÉVU

`uploads / utilisateur`

Les fichiers de notre compte.

`../`

remonter



AILLEURS SUR LE SERVEUR

`app.py · challenge.db`

Le code et la base de comptes.

`/download/…` Chemin contrôlé

`/raw?filename=…` Chemin non contrôlé

Schéma du comportement décrit dans le journal, pas une capture de l’application.

L’IDÉE À RETENIR

Le point faible : **laisser un nom de fichier sortir du dossier prévu.**

LE DÉCLIC · NE PAS S’ARRÊTER AU PREMIER TEXTE PLAUSIBLE

Un faux flag dans la base.
Une vraie piste juste à côté.

LE PIÈGE Un mot de passe ressemblait à un flag. LEURRE

01

Lire les comptes

La base contient aussi un compte administrateur.

→

02

Se connecter

Le tableau de bord admin montre d’autres fichiers.

→

03

Suivre l’indice

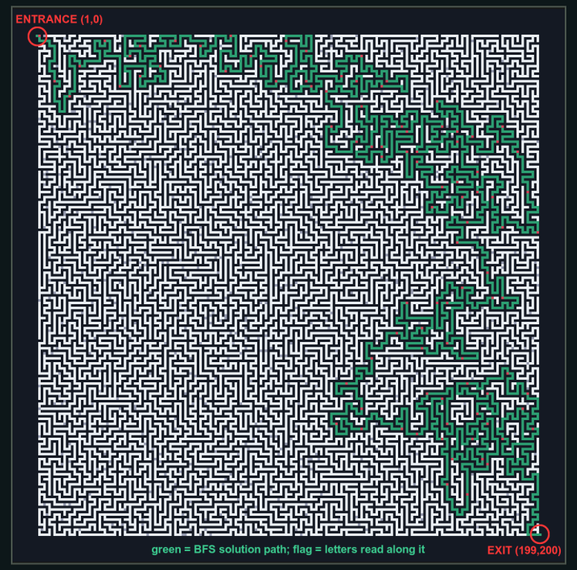
`flag.txt` indique que le flag est caché dans `flag.maze`.

L’IDÉE À RETENIR

Le bon chemin mène à **un labyrinthe de caractères.**

LA RÉSOLUTION · SUIVRE LE LABYRINTHE

Le chemin vers la sortie écrit le flag.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

201 × 201 cases dans le labyrinthe

1. Partir de l’entrée.
2. Explorer les passages accessibles.
3. Lire les lettres du chemin vers la sortie.

2 783 POSITIONS SUR LE CHEMIN

`BA26{tHis_Is_a_rEally_loNG_flag_…}`

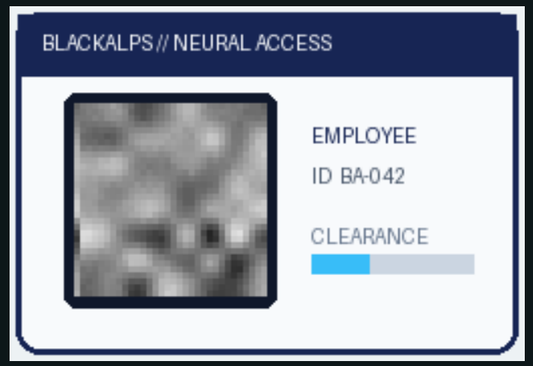
L’IDÉE À RETENIR

L’accès aux fichiers ouvre la porte. **Le parcours du labyrinthe donne la réponse.**

LE DÉFI · UNE IMAGE DÉCIDE DES DROITS

Pour entrer comme admin, il faut convaincre un modèle.

BADGE FOURNI



Classé « EMPLOYEE ».

→

LE SCANNER RENVOIE DES SCORES

EMPLOYEE	
INVALID	
ADMIN	≥ 99,5 %

Le seuil à dépasser pour obtenir l’accès.

L’IDÉE À RETENIR

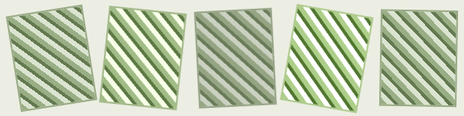
On peut modifier l’image et observer **le score renvoyé à chaque essai.**

LE DÉCLIC · UTILISER LE RETOUR DU MODÈLE

Changer quelques pixels.
Regarder dans quel sens aller.

01

Créer des variantes



De petites modifications autour du badge courant.

02

Comparer les scores

PLUS FROID | PLUS CHAUD

Quelles modifications rapprochent de « ADMIN » ?

03

Avancer et répéter

↻

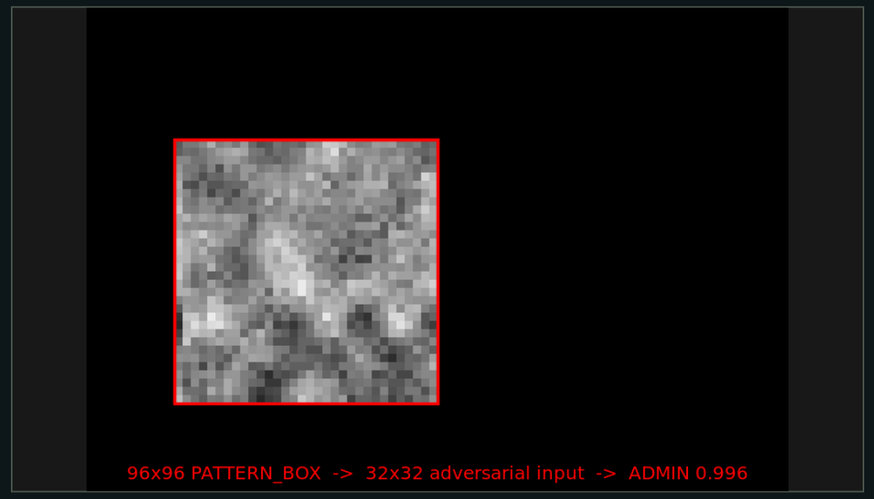
Ajuster l’image dans la direction estimée.

L’IDÉE À RETENIR

Les scores servent de guide, **sans connaître les poids du modèle.**

LE RÉSULTAT · LE MODÈLE ACCORDE L’ACCÈS

Ce badge ne ressemble pas à un admin. Le modèle l’accepte pourtant.



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

SCORE ADMIN RAPPORTÉ DANS LE JOURNAL

99,6 %

Seuil demandé : 99,5 %.

ACCÈS ACCORDÉ

`BA26{r3ached_1h3_t0P}`

L’IDÉE À RETENIR

Une décision d’accès fondée sur une image peut être **pilotée par les scores exposés.**

LE DÉFI · RETROUVER LE SECRET TRANSMIS PAR RADIO

La radio contient les messages.
Le firmware donne les règles.

ARTEFACT 01

10 s

d’enregistrement radio

Ce qu’une balance connectée
a envoyé à sa passerelle.

+

ARTEFACT 02

ESP32-C6

une copie du firmware

Le programme embarqué :
protocole et méthode de chiffrement.

L’IDÉE À RETENIR

Il faut faire parler **les deux fichiers ensemble.**

ÉTAPE 1 · DÉCODER LE SIGNAL

Avant de déchiffrer, il faut retrouver les paquets.

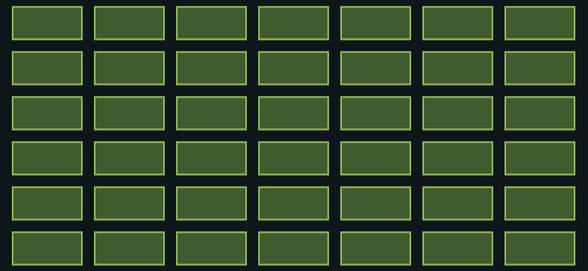
SIGNAL RADIO · SCHÉMA



Reconnaître les motifs utilisés par Zigbee.

→

PAQUETS RETROUVÉS



42 / 42

Contrôles d’erreur valides.

L’IDÉE À RETENIR

Le contrôle d’erreur confirme **que les octets ont été correctement décodés.**

ÉTAPE 2 · ÉCOUTER L’ENTRÉE DANS LE RÉSEAU

Au moment de rejoindre le réseau, la balance reçoit une clé.

LA CAPTURE CONTIENT L’ÉCHANGE D’APPAIRAGE

La clé réseau y est protégée à partir d’une clé par défaut connue.

01

Clé par défaut

`ZigBeeAlliance09`

Point de départ connu.

→

02

Clé réseau

Retrouvée dans l’échange qui accueille la balance.

→

03

6 fragments

Les rapports s’ouvrent.
Le contenu reste encore protégé.

L’IDÉE À RETENIR

Le premier verrou s’ouvre. **Il reste une couche propre à l’application.**

ÉTAPE 3 · RELIER LE MESSAGE À CETTE BALANCE

La dernière clé dépend de l’identité de la balance.

VU DANS LA RADIO

Identité de l’appareil

+

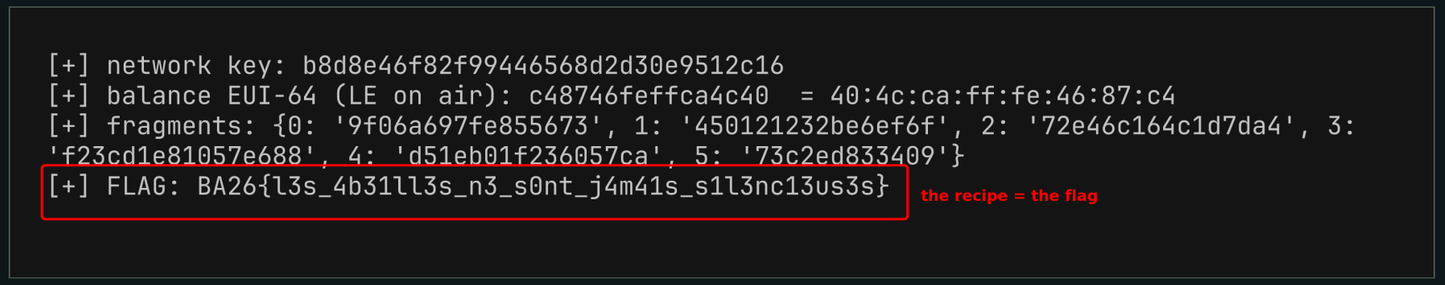
TROUVÉ DANS LE FIRMWARE

« propolis »

+

POUR CHAQUE MORCEAU

Numéro du fragment



PREUVE CONSERVÉE · CLIQUER POUR AGRANDIR

Déchiffrer chaque fragment avec RC4,
puis les assembler dans l’ordre.

`BA26{l3s_4b31ll3s_n3_s0nt_j4m41s_s1l3nc13us3s}`

L’IDÉE À RETENIR

La radio apporte l’identité et les données. **Le firmware explique la dernière couche.**

UNE RADIO DE FORMULE 1. UNE CONSIGNE.

Faire rentrer les 22 pilotes.

RADIO / COMMANDE CIBLE



“Box Box Box!”

MESSAGE EXACT — Un appel au stand pour chaque pilote.

01 **Le message**
Ce qu’on veut faire accepter.

02 **Une signature valide** (r, s)
La preuve que l’ordre est autorisé.

03 **La clé publique du pilote**
Elle vérifie la signature. Elle est dans le binaire.

LE VERROU

Connaître le message ne suffit pas.
Il faut une signature pour **chacun des 22 pilotes.**

NONCE = NOMBRE SECRET UTILISÉ POUR UNE SIGNATURE

Le nombre secret avait 255 valeurs possibles.

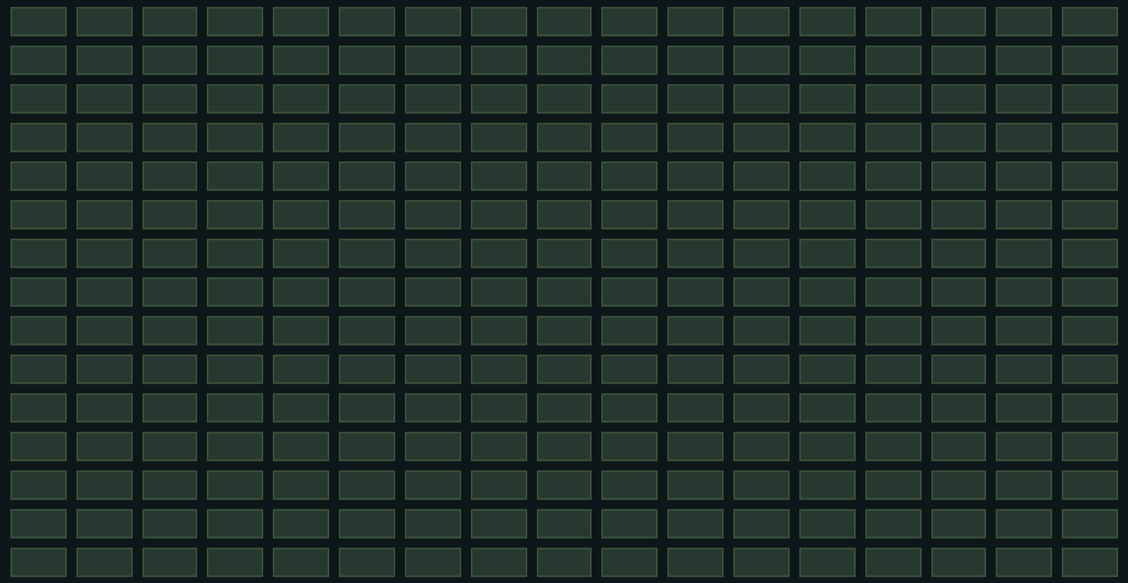
255

VALEURS POSSIBLES DE *k*

Un nombre compris entre 1 et 255.

On peut essayer toutes les valeurs.

TOUT L'ESPACE DE RECHERCHE 01 → 255



Essayer chaque valeur de k.

Calculer r, puis comparer au r observé.

`La même valeur de r ? Le nonce est retrouvé.`

CE QUE DONNE « SPY »

Le message est chiffré. **Sa signature (r, s) est visible.**

IL MANQUE ENCORE LE MESSAGE SIGNÉ.

Le texte était chiffré.
Mais facile à énumérer.

PREMIÈRE PISTE

Prédire l'horloge et le générateur.

Reproduire la suite de nombres pour retrouver le texte.

~~ABANDONNÉE DANS LE SOLVE FINAL~~

LA PISTE QUI A MARCHÉ

```
01 [pilote] Activate ECU mode {x}
02 [pilote] Go plan {L}{signe}{tour}
03 [pilote] Pit window lap {tour}, {pneu}
```

Exemples des modèles de messages du solveur.

≈ 800

MESSAGES CANDIDATS PAR PILOTE

Essayer chaque message possible, puis vérifier la clé obtenue.

Aucun besoin de deviner l'heure.

LES DEUX PETITES RECHERCHES SE REJOIGNENT.

Retrouver une clé.
Vérifier que c’est la bonne.

01 Signature observée + *02* Nonce retrouvé + *03* Un message possible

AVEC CES TROIS ÉLÉMENTS Calculer une **clé privée candidate.**

✓ ou ✗ **La clé obtenue correspond-elle au pilote ?**

On le vérifie avec sa clé publique, connue depuis le début.

✓ Le solveur a confirmé **22 clés privées sur 22.**

0 ÉCHEC

SIGNER « BOX BOX BOX! » AVEC CHAQUE CLÉ

Tout le plateau. Au stand.

22/22

COMMANDES ACCEPTÉES

01	02	03	04	05	06	07	08	09	10	11
12	13	14	15	16	17	18	19	20	21	22

LA PREUVE CONSERVÉE

```
[RADIO] VALTTERI BOTTAS -- BOX BOX BOX!  [22/22]

============================================================
!!  FULL GRID PIT CALL -- CHAOS IN THE PIT LANE  !!
============================================================

FLAG: BA26{T0t0_1ts_C4ll3d_4_M0t0r_R4c3_N0t_S3cur3_Crypt0}
```

CAPTURE ANNOTÉE DU RÉSULTAT

↗ AGRANDIR

« FULL GRID PIT CALL — CHAOS IN THE PIT LANE »

Solve : **Claude** · Résultat enregistré dans `final_resp.txt`

À RETENIR

Un nonce minuscule + des messages prévisibles **ont suffi à récupérer les clés de signature.**

LE DÉFI · UN MESSAGE CACHÉ DANS UNE MÉMOIRE

La caméra cachait un secret. On a fouillé ses tiroirs.

LE POINT DE DÉPART

Tapo C500

Une caméra, un petit ordinateur.

Sa mémoire contient des programmes, des réglages et des fichiers.

→

LE FICHIER À EXPLORER

Le dump

Une copie de la mémoire.

Le but : retrouver un texte au format BA26{…}.

L'IDÉE À RETENIR

Chercher « BA26 » ne suffit pas : **le texte est compressé ou chiffré.**

LE DÉCLIC · UNE ZONE UTILE AVAIT ÉCHAPPÉ À L’EXTRACTION

Le livre avait perdu sa couverture. Pas ses pages.

01

Un début illisible

L’outil ne reconnaît pas le début d’un système de fichiers.

→

02

Des indices intacts

Des blocs compressés contiennent encore la position et la taille des fichiers.

→

03

Une récupération ciblée

On utilise ces indications pour extraire les fichiers qui nous intéressent.

L’IDÉE À RETENIR

L’outil n’avait pas tout vu. **La mémoire avait encore des choses à raconter.**

LES INDICES · TROIS PIÈCES QUI SE COMPLÈTENT

Le message était secret.
Le mode d’emploi était à côté.

01

flag.enc

Le message chiffré.
35 octets dans le fichier fourni.

\+

02

ctf_unlock

Le programme qui explique comment retrouver le texte.

\+

03

devId

L’identifiant de la caméra, présent dans ses réglages.

L’IDÉE À RETENIR

Le démarrage relie les indices : **identifiant + programme + message chiffré.**

LE MÉCANISME · REPRODUIRE LA RECETTE

Pour enlever le masque, on refait le même masque.

01

Les ingrédients

Identifiant de la caméra
+ texte fixe « C500-CTF-v1 »
+ compteur.

→

02

Fabriquer le masque

SHA-256 produit une suite
de nombres reproductible
avec ces mêmes ingrédients.

→

03

Retrouver le texte

L'opération XOR combine
le message chiffré et ce masque.
Le texte réapparaît.

L'IDÉE À RETENIR

Pas besoin de deviner le flag : on reproduit la recette du programme.

LE RÉSULTAT · TEXTE CONSERVÉ DANS FLAG.TXT

Souriez.
Le flag est sorti.

CONTENU EXACT DU FICHIER FLAG.TXT FOURNI

`BA26{a-R00t-0f-Tru5t-Sh1ps-N0-K3ys}`

35 caractères · même résultat que dans le PowerPoint.

ÉTAT DOCUMENTÉ **Flag validé.** Soumission confirmée par la plateforme.

L'IDÉE À RETENIR

Le secret partageait sa mémoire avec **sa recette et les données nécessaires pour le lire.**

DU RÉSULTAT AU RAISONNEMENT

Le flag termine la recherche.
La preuve raconte le solve.

01	Identifier le verrou	Qu’est-ce que le système vérifie vraiment ?
02	Trouver l’hypothèse fragile	Un identifiant, un score ou des nombres suffisent-ils ?
03	Confirmer le résultat	Une sortie conservée, un test de clé, un flag réel.

L’IDÉE À RETENIR

Les agents ont trouvé les chemins. **Nous pouvons maintenant les expliquer.**